where U and R_D are given while R_D^* is a learned constant. If no repeats are done, the second part of the sum is zero and the term simplifies to the single-epoch scaling laws from Equation 7. While $R_D \ll R_D^*$, the second term is approximated as $U \cdot R_D$ and for $R_D \gg R_D^*$, it plateaus at $U \cdot R_D^*$. Hence R_D^* corresponds to the number of times we can repeat tokens before seeing sharply diminishing returns.

Let us consider a concrete example to show that Equation 13 is a very good approximation of Equation 10 and make the equations more intuitive. Suppose repeated data retains 75% of its value ($\delta = 0.25$) and we train on a single token or data unit ($U = 1$) for five epochs, i.e. we repeat it four times ($R_D = 4$). In that case Equation 10 yields $D' = U + (1-\delta)U\frac{(1-(1-\delta)^{R_D})}{\delta} = 1 + (0.75) * 4 * (1 - 0.75^4) = 3.05$. Thus despite training for 5 total units (4 of which are repetitions), we only get the value equivalent to 3.05 units. As we have defined $R_D^* = (1-\delta)/\delta$, the corresponding R_D^* value is 3. Setting $R_D^* = 3$ in Equation 13 yields $D' = U + U \cdot R_D^* \cdot (1 - e^{-R_D/R_D^*}) = 1 + 3 * (1 - e^{-4/3}) = 3.21$. Due to our approximations, the results are not the same, i.e. 3.21 is slightly higher than 3.05. However, note that the data term is additionally raised to a power of $\beta = 0.353$ (see Equation 7; Appendix B), thus the actual difference calculated as $((3.21^{0.353})/(3.05^{0.353})) - 1$ is a mere 1.8% despite this relatively large δ of 0.25. Equation 13 has the benefit that we can interpret R_D^* as the number of repetitions beyond which repeating yields sharply diminishing returns and flattens out soon after. Consider $R_D = 100$ then $D' = 1 + 3 * (1 - e^{-100/3}) = 3.99$. No matter how many repeats are done the effective data will never exceed 4 i.e. it plateaus at $U + R_D^* U$ as R_D tends to infinity.

Similarly, we consider repeating parameters. Symmetric to seeing the same data, excess parameters learn the same features and do not add any value in the extreme. For the Chinchilla equation (Equation 7) increasing parameters from 1 billion to 10 billion yields the same absolute decrease in loss regardless of whether the dataset is a single token or 1 billion tokens. However, intuition and our data (Appendix F) suggest that in the first case, adding parameters should not decrease loss at all, as the additional 9 billion parameters cannot possibly learn anything from the single token that the first 1 billion parameters have not already learned. Thus, to allow excess parameters to decay to adding nothing, we also replace N with a symmetric version of Equation 13 yielding our final equation:

$$L(U_N, U_D, R_N, R_D) = \frac{A}{(U_N + U_N R_N^*(1 - e^{\frac{-R_N}{R_N^*}}))^\alpha} + \frac{B}{(U_D + U_D R_D^*(1 - e^{\frac{-R_D}{R_D^*}}))^\beta} + E \tag{14}$$

We define U_N, as the number of "unique" parameters that provide an optimal fit for U_D. Additional parameters decay with a symmetric version of the expression for repeated data. R_N is the number that the "unique" parameters are repeated i.e. $R_N = \max\{(N/U_N) - 1, 0\}$. If $R_N^* = \infty$, additional parameters do not decay at all and $(U_N + U_N R_N^*(1 - e^{\frac{-R_N}{R_N^*}}))$ reduces to N. We compute U_N from U_D by setting $D_{opt} = U_D$ and rearranging Equation 3 to map from D_{opt} to N_{opt}. U_N is then $\min\{N_{opt}, N\}$. This is equivalent to the following:

$$U_N = \min\{((U_D \cdot G)^{\beta/\alpha}) \cdot G, N\} \quad \text{where} \quad G = \left(\frac{\alpha A}{\beta B}\right)^{\frac{1}{\alpha+\beta}} \tag{15}$$

Equation 14 is a generalization of Equation 7: It provides the same estimates for optimal model and data size in the single epoch case, but allows for decay in the value of parameters and tokens, thus generalizing to training for multiple epochs and with excess parameters. It can thus be used as a direct replacement of Equation 7. If R_N^* and R_D^* are unknown, one can simply set them to infinity by default, which will make Equation 14 completely equivalent to Equation 7.

To learn the parameters R_N^* and R_D^*, we largely follow the approach from [42]. We fix a, b, e, α, β to the values learned on C4 in Appendix B and minimize:

$$\min_{R_N^*, R_D^*} \sum_{\text{Run } i} \text{Huber}_\delta\Big(\text{LSE}\big(a - \alpha \log(U_N^i + U_N^i R_N^*(1 - e^{\frac{-R_N^i}{R_N^*}})), \\ b - \beta \log(U_D^i + U_D^i R_D^*(1 - e^{\frac{-R_D^i}{R_D^*}})), e\big) - \log L^i\Big) \tag{16}$$

We use the LBFGS algorithm to find local minima of the objective above, started on a grid of initialization given by: $R_N^* \in \{0., 4., \ldots, 20.\}$ and $R_D^* \in \{0., 4., \ldots, 20.\}$. We fit on 182 samples with parameters varying from 7 million up to 9 billion and epochs ranging from 1 to 500. We removed outliers referenced in Appendix F from our fitting, as our formulas do not allow for excess parameters or excess epochs to negatively impact performance. We assume excess parameters or epochs only cause performance to plateau but never to worsen. However, it is difficult to identify all samples where excess parameters or epochs hurt, as for some data budgets we only train a single model, thus we do not know if the loss of that model is already in the range where it starts to increase again. Further, there are samples where loss initially increases and then decreases as a function of epochs (double descent, see Appendix D), which further contributes to noise in the fitting. Nevertheless, we are able to get a fairly stable fit resulting in $R_N^* = 5.309743$ and $R_D^* = 15.387756$. Since $R_D^* > R_N^*$, excess parameters decay faster. Hence, the data-constrained efficient frontiers in Figures 1,3 suggest scaling compute allocated to epochs faster than to parameters. This value of R_D^* yields $\delta \approx 6 * 10^{-2}$ (0.19 for R_N^*), which respects the assumption that δ is small. Inserting these learned parameters and the parameters from Appendix B, and simplifying Equation 15 yields the precise formulation we use to predict loss (L) given unique tokens (U_N), parameter repetitions (R_N) and data repetitions (R_D):

$$L(U_D, R_N, R_D) = \frac{521}{(U_N + 5.3 \cdot U_N(1 - e^{\frac{-R_N}{5.3}}))^{0.35}} + \frac{1488}{(U_D + 15.4 \cdot U_D(1 - e^{\frac{-R_D}{15.4}}))^{0.35}} + 1.87$$

$$\text{where } U_N = U_D \cdot 0.051 \tag{17}$$

Table 1: **Comparison of different versions of our parametric fit.** All versions are fitted on the same 182 samples. We report the fitting loss and the R^2 (coefficient of determination) of the predicted loss compared to the actual loss. No decay corresponds to assuming Chinchilla holds for repeated data without modification necessary. For Equation 10, we use the same equation for D and N renaming the δ to R_D^* and R_N^*.

Parametric Fit	R_D^*	R_N^*	Loss (↓)	R^2 (↑)
No decay	-	-	-	0.4452
Equation 14 but only decay N	-	713.0015	0.0241	0.4488
Equation 14 but only decay D	2.9157	-	0.0169	0.7354
Equation 14	15.3878	5.3097	0.0158	0.7722
Equation 10 for both N and D	0.0104	0.3676	0.0155	0.7988
Equation 18 for both N and D	0.0105	0.3676	0.0155	0.7987
Equation 22	26530.611	2040.8163	0.0596	0.5110

We experiment with different versions of our formula and display the learned values in Table 1. No decay or decaying only D or N of Equation 14 leads to worse loss and R^2 than Equation 14. Thus, it is important to decay both the value of excess parameters and data repetitions. We also consider an explicit exponential where $D' = \sum_{k=0}^{R_D} U * e^{-R_D^* k}$, hence from Equation 9 it follows:

$$D' = U \frac{1 - (e^{-R_D^*})^{R_D + 1}}{1 - e^{-R_D^*}} \tag{18}$$

This explicit decay, Equation 10, and Equation 14 all yield similar results with R^2 around 80. Equation 14 fits the data slightly worse than Equation 10, likely due to our approximations. Nevertheless, we use Equation 14 throughout as it has fewer terms, and we find it easier to interpret.

A.1 Analytical properties of compute-optimal point

In our case, consider the setting of a fixed compute budget C and a fixed budget of unique tokens U_D implying a set of unique parameters U_N. Let R_D denote the number of times we repeat data (we assume that we are in the multi-epoch regime and hence $R_D > 0$).

Write $U_D = cU_N$ (for Chinchilla $c \approx 20$). When $R_D \ll R_D^*$ and $R_N \ll R_N^*$, our scaling agrees with Chinchilla, and so the point (U_N, U_D), corresponding to $R_D = R_N = 0$ is on the optimal

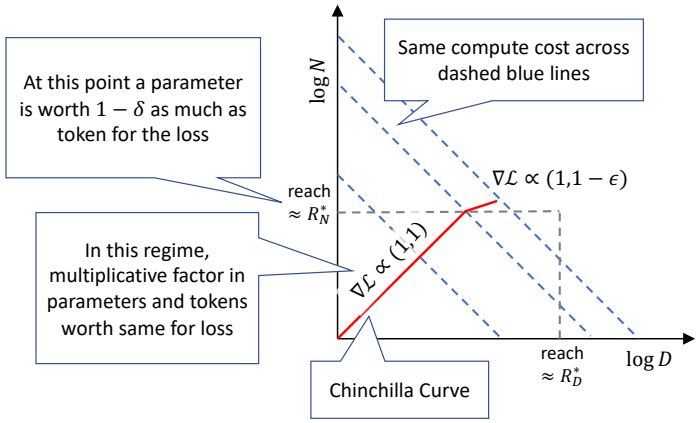

Figure 7: A cartoon of how the compute-optimal tradeoff deviates from Chinchilla as we increase the number of epochs. Initially the model size and tokens processed grow proportionally ($R_N = R_D$) but since $R_N^* < R_D^*$, at some point adding parameters offers worse returns compared to increasing the number of tokens processed, and hence we deviate from the Chinchilla curve.

compute curve. Increasing R_D by ϵ corresponds to increasing the number of tokens by $\epsilon U_D = \epsilon c U_N$, while increasing R_N by ϵ corresponds to increasing the number of parameters by ϵU_N. For small positive R_D, R_N, our curve agrees with Chinchilla and so we need to increase R_N, R_D by the same amount to maintain the proportionality. Hence up to some value $r > 0$, the optimal compute curve corresponds to $R_N = R_D = r$. Our curve differs from Chinchilla when r gets closer to either R_N^* or R_D^*. At this point, we start to see sharply diminishing returns.

In our setting, $R_D^* > R_N^*$ which means that we reach the point $r \approx R_N^*$ first. At this point, each added parameter is worth less (specifically worth e^{-r/R_N^*}), than an added data point, despite them having equal computational cost. Hence processing more tokens will be more effective than increasing the number of parameters, and we expect the optimal compute curve to break away from proportionality. This is indeed what we see.

B C4 Scaling Coefficients

While Hoffmann et al. [42] have shown that the equal scaling of model parameters and training tokens holds across different training datasets, the precise ratios vary considerably across datasets and approaches. For example given the Gopher [89] compute budget of 5.76×10^{23} FLOPs, their parametric loss function fitted on MassiveWeb predicts an optimal allocation of 40 billion parameters. Meanwhile, if the training dataset is C4 [90] their IsoFLOP approach predicts 73 billion parameters to be optimal, almost twice as much. However, for C4, which is our training dataset, they do not provide the coefficients necessary to compute loss with their parametric loss function. Based on their IsoFLOP training runs on C4, they only provide the information that for C4, compute (C) allocated to data (D) and parameters (N) should be scaled *exactly* equally for optimality, i.e. $a = b = 0.5$ in the relationship $N_{opt} \propto C^a$ and $D_{opt} \propto C^b$. This corresponds to $\alpha = \beta$ in the parametric loss function (Equation 2). Thus, we use this information together with the methodology and C4 data points from [42] to fit the parametric loss function. We tie the parameters α and β to be equal and optimize

$$\min_{a,b,e,\alpha,\beta} \sum_{\text{Run } i} \text{Huber}_\delta\Big(\text{LSE}\big(a - \alpha \log N_i, b - \beta \log D_i, e\big) - \log L_i\Big) \tag{19}$$

where LSE is the log-sum-exp operator and N_i, D_i and L_i the model size, dataset size and loss of the ith run, and $\delta = 10^{-3}$. We fit on 54 samples on a grid of initialization given by: $\alpha \in \{0., 0.5, \ldots, 2.\}$, $\beta \in \{0., 0.5, \ldots, 2.\}$, $e \in \{-1., -.5, \ldots, 1.\}$, $a \in \{0, 5, \ldots, 25\}$, and $b \in \{0, 5, \ldots, 25\}$. Our fit results in $a = 6.255414$, $b = 7.3049974$, $e = 0.6254804$, $\alpha = \beta = 0.3526596$. Exponentiating a, b and e to get A, B and E and inserting all learned coefficients into Equation 2 then allows us to compute loss (L) as a function of parameters and data: